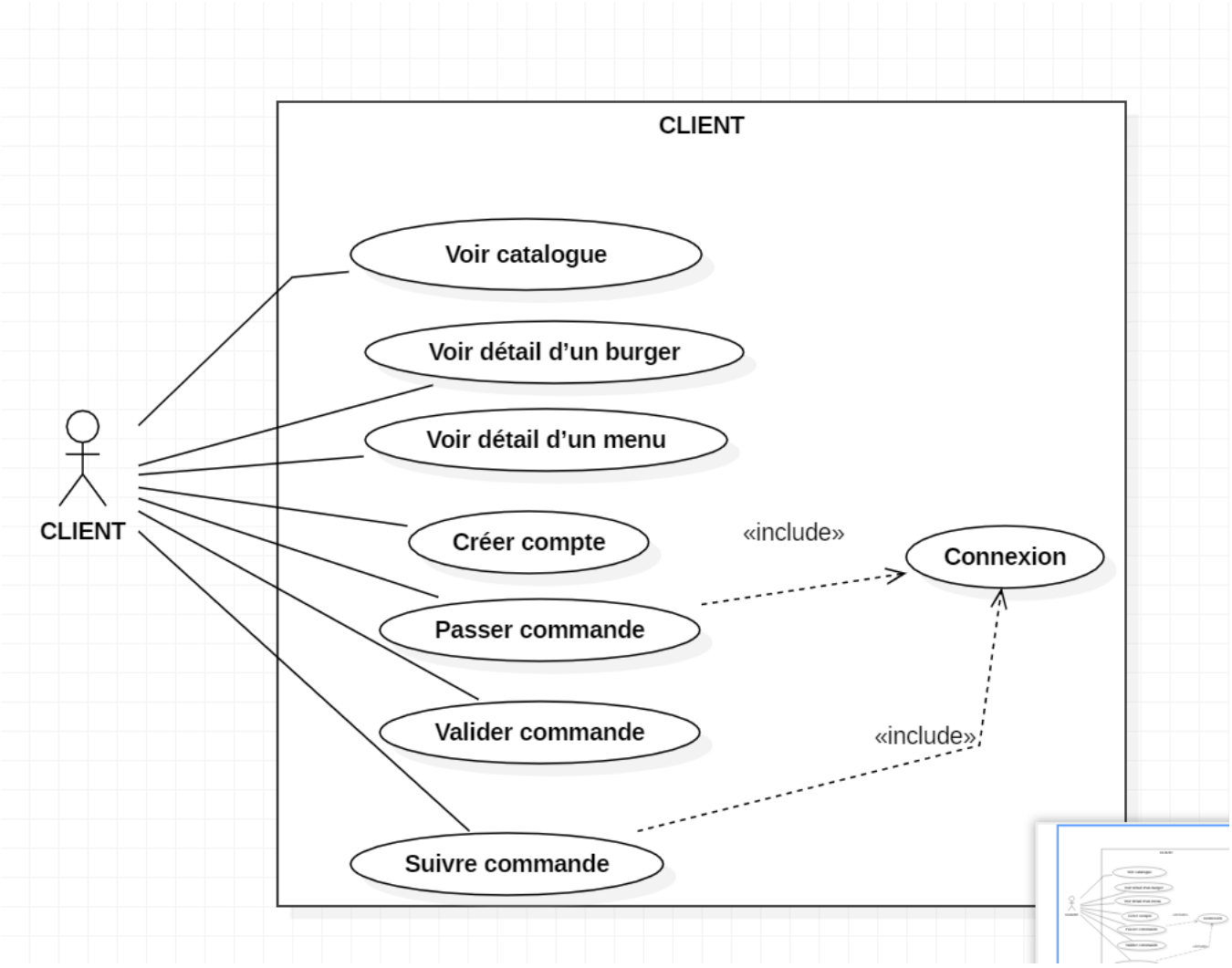
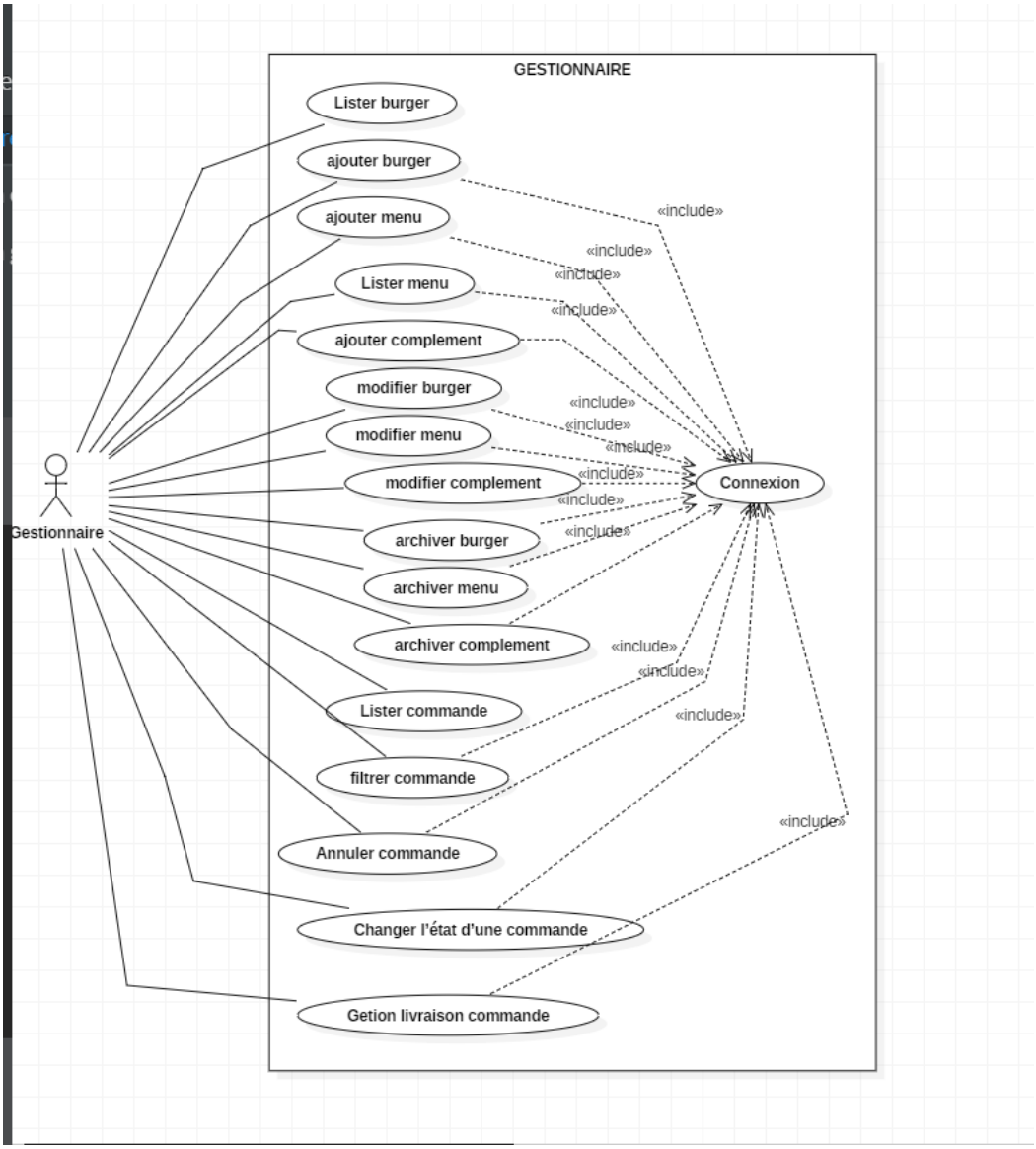
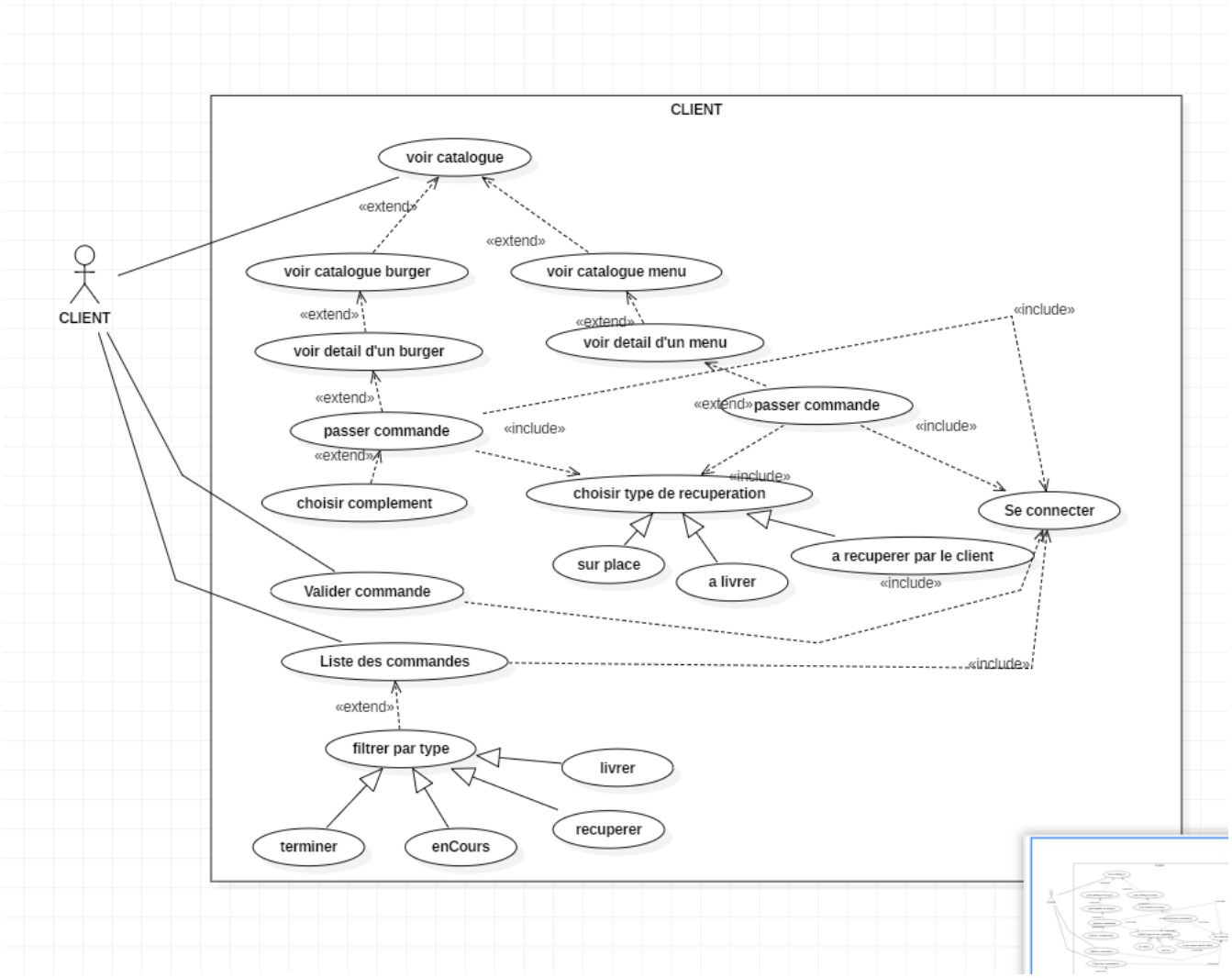
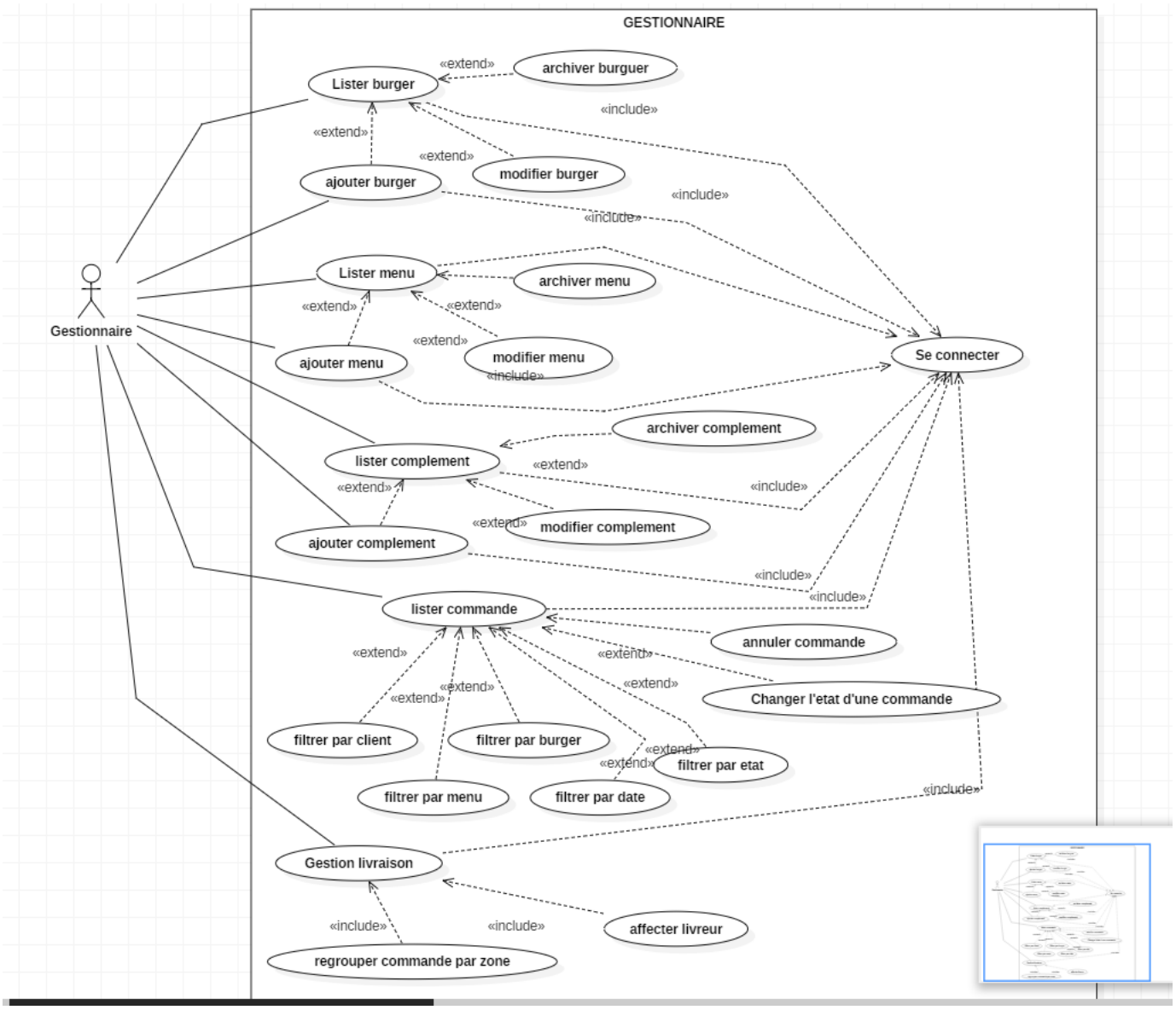
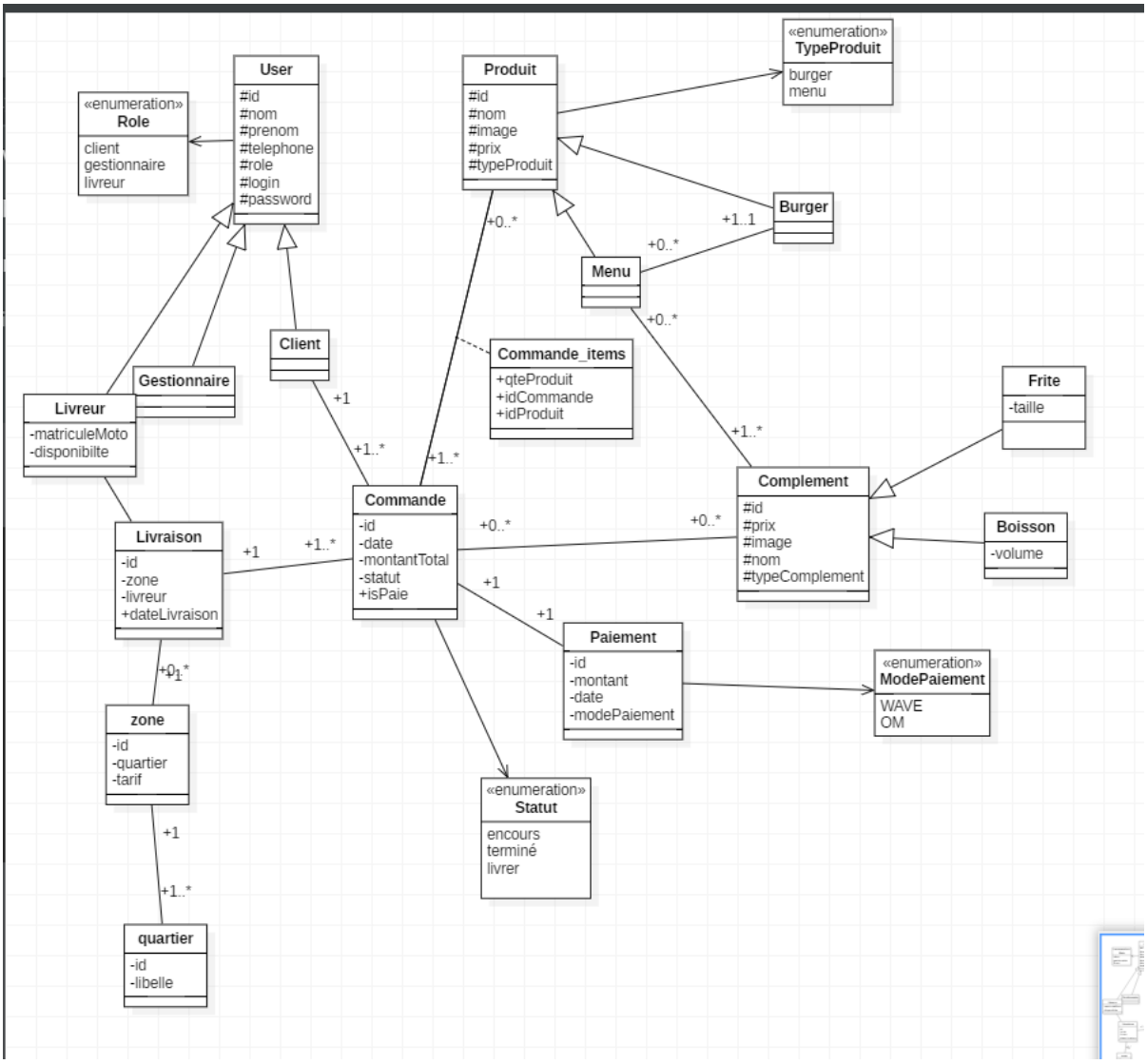
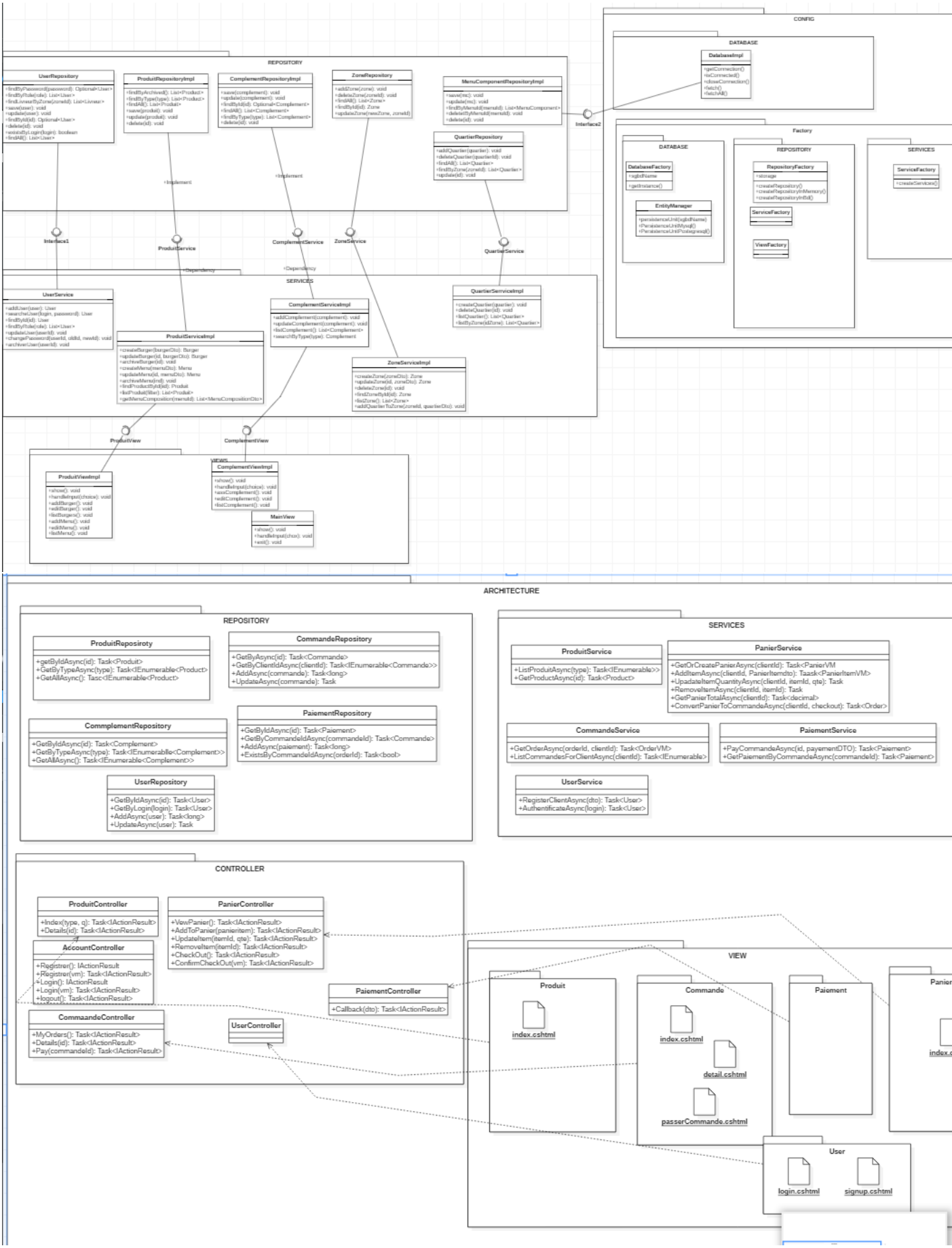

CLIENT
Voir catalogue
Voir détail d'un burger
Voir détail d'un menu
Créer compte
«include»
Connexion
Passer commande
Valider commande
«include»
Suivre commande
CLIENT

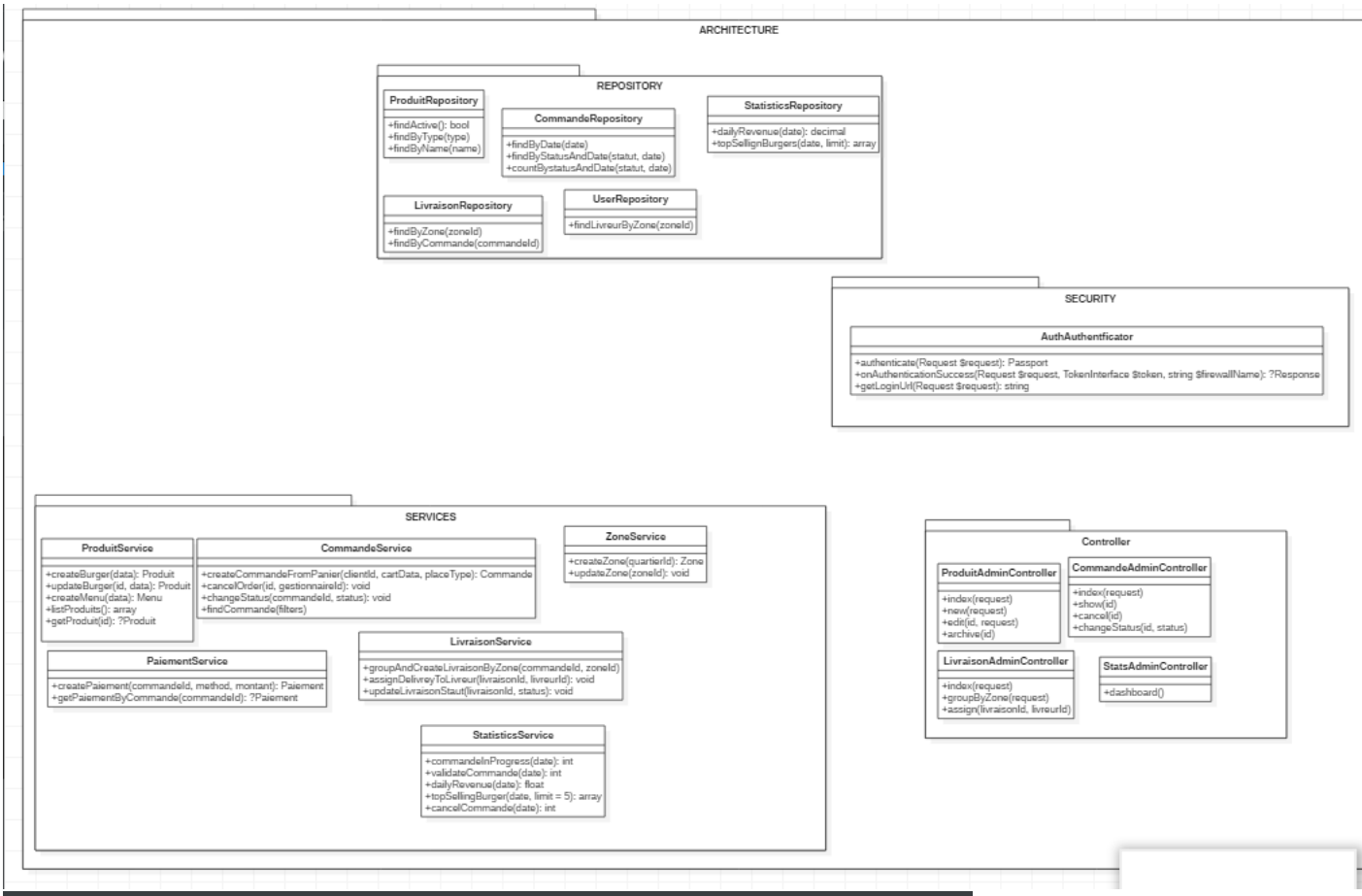

GESTIONNAIRE
Lister burger
ajouter burger
ajouter menu
Lister menu
ajouter complement
modifier burger
modifier menu
modifier complement
archiver burger
archiver menu
archiver complement
Lister commande
filtrer commande
Annuler commande
Changer l'état d'une commande
Getion livraison commande
Connexion
Gestionnaire
«include»

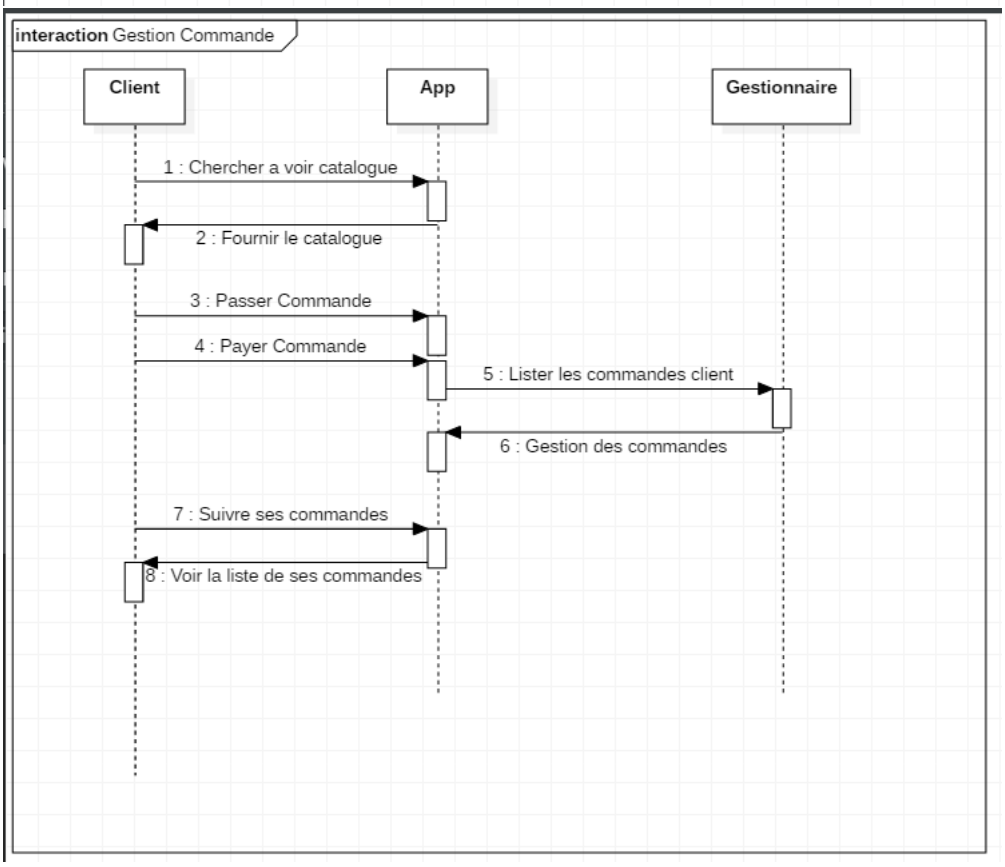

CLIENT
voir catalogue
«extend»
«extend»
voir catalogue burger
voir catalogue menu
CLIENT
«extend»
«extend»
voir detail d'un burger
voir detail d'un menu
«include»
«extend»
«extend»
passer commande
passer commande
«include»
«include»
«extend»
«include»
choisir complement
choisir type de recuperation
Se connecter
sur place
a livrer
a recuperer par le client
«include»
Valider commande
Liste des commandes
«include»
«extend»
filtrer par type
livrer
recuperer
terminer
enCours


GESTIONNAIRE
Gestionnaire
Lister burger
archiver burguer
ajouter burger
modifier burger
Lister menu
archiver menu
ajouter menu
modifier menu
Se connecter
archiver complement
lister complement
ajouter complement
modifier complement
lister commande
annuler commande
Changer l'etat d'une commande
filtrer par client
filtrer par burger
filtrer par etat
filtrer par menu
filtrer par date
Gestion livraison
affecter livreur
regrouper commande par zone
«extend»
«include»


«enumeration»
TypeProduit
burger
menu
User
#id
#nom
#prenom
#telephone
#role
#login
#password
«enumeration»
Role
client
gestionnaire
livreur
Produit
#id
#nom
#image
#prix
#typeProduit
Burger
Menu
Client
Gestionnaire
Livreur
-matriculeMoto
-disponibilte
Commande_items
+qteProduit
+idCommande
+idProduit
Frite
-taille
Complement
#id
#prix
#image
#nom
#typeComplement
Boisson
-volume
Commande
-id
-date
-montantTotal
-statut
+isPaie
Livraison
-id
-zone
-livreur
+dateLivraison
Paiement
-id
-montant
-date
-modePaiement
«enumeration»
ModePaiement
WAVE
OM
zone
-id
-quartier
-tarif
«enumeration»
Statut
encours
terminé
livrer
quartier
-id
-libelle
+0..*
+1..1
+1
+1..*


REPOSITORY
UserRepository
+findByPassword(password): Optional<User>
+findByRole(role): List<User>
+findLivreurByZone(zoneId): List<Livreur>
+save(user): void
+update(user): void
+findById(id): Optional<User>
+delete(id): void
+existsByLogin(login): boolean
+findAll(): List<User>
ProduitRepositoryImpl
+findByArchived(): List<Produit>
+findByType(type): List<Produit>
+findAll(): List<Produit>
+save(produit): void
+update(produit): void
+delete(id): void
ComplementRepositoryImpl
+save(complement): void
+update(complement): void
+findById(id): Optional<Complement>
+findAll(): List<Complement>
+findByType(type): List<Complement>
+delete(id): void
ZoneRepository
+addZone(zone): void
+deleteZone(zoneId): void
+findAll(): List<Zone>
+findById(id): Zone
+updateZone(newZone, zoneId)
MenuComponentRepositoryImpl
+save(mc): void
+update(mc): void
+findByMenuId(menuId): List<MenuComponent>
+deleteByMenuId(menuId): void
+delete(id): void
QuartierRepository
+addQuartier(quartier): void
+deleteQuartier(quartierId): void
+findAll(): List<Quartier>
+findByZone(zoneId): List<Quartier>
+update(id): void
Interface2
CONFIG
DATABASE
DatabaseImpl
+getConnection()
+isConnected()
+closeConnection()
+fetch()
+fetchAll()
Factory
DATABASE
DatabaseFactory
+sqliteName
+getInstance()
EntityManager
+persistenceUnitSqliteName()
+PersistenceUnitMysql()
+PersistenceUnitPostgresql()
REPOSITORY
RepositoryFactory
+storage
+createRepository()
+createRepositoryInMemory()
+createRepositoryInfile()
ServiceFactory
ViewFactory
SERVICES
ServiceFactory
+createServices()
+Implement
+Implement
Interface1
ProduitService
ComplementService
ZoneService
QuartierService
+Dependency
+Dependency
SERVICES
UserService
+addUser(user): User
+searchUser(login, password): User
+findById(id): User
+findByRole(role): List<User>
+updateUser(user,id): void
+changePassword(userId, oldId, newId): void
+archiverUser(userId): void
ProduitServiceImpl
+createBurger(burgerDto): Burger
+updateBurger(id, burgerDto): Burger
+archiveBurger(id): void
+createMenu(menuDto): Menu
+updateMenu(id, menuDto): Menu
+archiveMenu(id): void
+findProductById(id): Produit
+findProduit(filter): List<Produit>
+getMenuComposition(menuId): List<MenuCompositionDto>
ComplementServiceImpl
+addComplement(complement): void
+updateComplement(complement): void
+listComplement(): List<Complement>
+searchByType(type): Complement
QuartierServiceImpl
+createQuartier(quartier): void
+deleteQuartier(id): void
+listQuartier(): List<Quartier>
+listByZone(idZone): List<Quartier>
ZoneServiceImpl
+createZone(zoneDto): Zone
+updateZone(id, zoneDto): Zone
+deleteZone(id): void
+findZoneById(id): Zone
+listZone(): List<Zone>
+addQuartierToZone(zoneId, quartierDto): void
ProduitView
ComplementView
VIEWS
ProduitViewImpl
+show(): void
+handleInput(choice): void
+addBurger(): void
+editBurger(): void
+listBurgers(): void
+addMenu(): void
+editMenu(): void
+listMenu(): void
ComplementViewImpl
+show(): void
+handleInput(choice): void
+saveComplement(): void
+editComplement(): void
+listComplement(): void
MainView
+show(): void
+handleInput(choix): void
+exit(): void
ARCHITECTURE
REPOSITORY
ProduitReposiroty
+getByIdAsync(id): Task<Produit>
+GetByTypeAsync(type): Task<IEnumerable<Product>
+GetAllAsync(): Tasc<IEnumerable<Product>
CommandeRepository
+GetByAsync(id): Task<Commande>
+GetByClientIdAsync(clientId): Task<IEnumerable<Commande>>
+AddAsync(commande): Task<long>
+UpdateAsync(commande): Task
CommplementRepository
+GetByIdAsync(id): Task<Complement>
+GetByTypeAsync(type): Task<IEnumerablle<Complement>>
+GetAllAsync(): Task<IEnumerable<Complement>>
PaiementRepository
+GetByIdAsync(id): Task<Paiement>
+GetByCommandeIdAsync(commandeId): Task<Commande>
+AddAsync(paiement): Task<long>
+ExistsByCommandeIdAsync(orderId): Task<bool>
UserRepository
+GetByIdAsync(id): Task<User>
+GetByLogin(login): Task<User>
+AddAsync(user): Task<long>
+UpdateAsync(user): Task
SERVICES
ProduitService
+ListProduitAsync(type): Task<IEnumerable>>
+GetProductAsync(id): Task<Product>
PanierService
+GetOrCreatePanierAsync(clientId): Task<PanierVM
+AddItemAsync(clientId, PanierItemdto): Taask<PanierItemVM>
+UpadateItemQuantityAsync(clientId, itemId, qte): Task
+RemoveItemAsync(clientId, itemId): Task
+GetPanierTotalAsync(clientId): Task<decimal>
+ConvertPanierToCommandeAsync(clientId, checkout): Task<Order>
CommandeService
+GetOrderAsync(orderId, clientId): Task<OrderVM>
+ListCommandesForClientAsync(clientId): Task<IEnumerable>
PaiementService
+PayCommandeAsync(id, paysmentDTO): Task<Paiement>
+GetPaiementByCommandeAsync(commandeId): Task<Paiement>
UserService
+RegisterClientAsync(dto): Task<User>
+AuthentificateAsync(login): Task<User>
CONTROLLER
ProduitController
+Index(type, q): Task<IActionResult>
+Details(id): Task<IActionResult>
PanierController
+ViewPanier(): Task<IActionResult>
+AddToPanier(panieritem): Task<IActionResult>
+UpdateItem(itemId, qte): Task<IActionResult>
+RemoveItem(itemId): Task<IActionResult>
+CheckOut(): Task<IActionResult>
+ConfirmCheckOut(vm): Task<IActionResult>
AccountController
+Registrer(): IActionResult
+Registrer(vm): Task<IActionResult>
+Login(): IActionResult
+Login(vm): Task<IActionResult>
+logout(): Task<IActionResult>
PaiementController
+Callback(dto): Task<IActionResult>
CommaandeController
+MyOrders(): Task<IActionResult>
+Details(id): Task<IActionResult>
+Pay(commandeId): Task<IActionResult>
UserController
VIEW
Produit
index.cshtml
Commande
index.cshtml
detail.cshtml
passerCommande.cshtml
Paiement
Panier
index.c
User
login.cshtml
signup.cshtml


ARCHITECTURE
REPOSITORY
ProduitRepository
+findActive(): bool
+findByType(type)
+findByName(name)
CommandeRepository
+findByDate(date)
+findByStatusAndDate(statut, date)
+countBystatusAndDate(statut, date)
StatisticsRepository
+dailyRevenue(date): decimal
+topSellignBurgers(date, limit): array
LivraisonRepository
+findByZone(zoneId)
+findByCommande(commandeId)
UserRepository
+findLivreurByZone(zoneId)
SECURITY
AuthAuthentficator
+authenticate(Request $request): Passport
+onAuthenticationSuccess(Request $request, TokenInterface $token, string $firewallName): ?Response
+getLoginUrl(Request $request): string
SERVICES
ProduitService
+createBurger(data): Produit
+updateBurger(id, data): Produit
+createMenu(data): Menu
+listProduits(): array
+getProduit(id): ?Produit
CommandeService
+createCommandeFromPanier(clientId, cartData, placeType): Commande
+cancelOrder(id, gestionnaireId): void
+changeStatus(commandeId, status): void
+findCommande(filters)
ZoneService
+createZone(quartierId): Zone
+updateZone(zoneId): void
PaiementService
+createPaiement(commandeId, method, montant): Paiement
+getPaiementByCommande(commandeId): ?Paiement
LivraisonService
+groupAndCreateLivraisonByZone(commandeId, zoneId)
+assignDelivreyToLivreur(livraisonId, livreurId): void
+updateLivraisonStaut(livraisonId, status): void
StatisticsService
+commandeInProgress(date): int
+validateCommande(date): int
+dailyRevenue(date): float
+topSellingBurger(date, limit = 5): array
+cancelCommande(date): int
Controller
ProduitAdminController
+index(request)
+new(request)
+edit(id, request)
+archive(id)
CommandeAdminController
+index(request)
+show(id)
+cancel(id)
+changeStatus(id, status)
LivraisonAdminController
+index(request)
+groupByZone(request)
+assign(livraisonId, livreurId)
StatsAdminController
+dashboard()


interaction Gestion Commande
Client
App
Gestionnaire
1 : Chercher a voir catalogue
2 : Fournir le catalogue
3 : Passer Commande
4 : Payer Commande
5 : Lister les commandes client
6 : Gestion des commandes
7 : Suivre ses commandes
8 : Voir la liste de ses commandes